SIMATS ENGINEERING

Engineer to Excel

SIMATS

Saveetha Institute of Medical And Technical Sciences

(Declared as Deemed to be University under Section 3 of UGC Act 1956)

Department of Computer Science and Engineering

Course Code & Name	CSA06 – Design and Analysis of Algorithms
Assessment	CO1 – Assessment Tool 1: IT Application-Based Problem Solving – Scalability Audit of an E-Commerce Platform
Course Outcome Assessed	Determine algorithm efficiency using asymptotic notations and mathematical techniques including Master Theorem and substitution method.
Duration	3 Days
Mode	Individual
Student Reg. No.	**KAVIYA.G**
Student Name	**192524281**

Instructions: Answer all parts. Show the basic operation, every step of each derivation and the final asymptotic result, and interpret what each result means for the platform. Assume the platform's servers execute 10^8 basic operations per second.

IT APPLICATION-BASED PROBLEM

Scenario – Scalability Audit of "ShopSphere", an Online Shopping Platform: ShopSphere is a growing e-commerce platform whose product catalogue currently holds n = 100,000 products and is expected to reach n = 1,000,000 within a year. Before the expansion, the CTO has asked you, as the platform's algorithm engineer, to audit five back-end modules, predict how each will behave as n grows, and recommend which modules must be redesigned. The modules are described below.

Module M1 – Duplicate-Listing Detector: Sellers sometimes upload the same product twice. Every night, the following routine compares every product with every later product.

```
FOR i ← 0 TO n − 2 DO
   FOR j ← i + 1 TO n − 1 DO
      IF Product[i].title = Product[j].title AND Product[i].seller =
Product[j].seller THEN
         MarkDuplicate(j)
      END IF
   END FOR
END FOR
```

Module M2 – API Rate Limiter: The public API audits request windows of doubling size using the routine below. A team member claims that it is Θ(n log n) because the outer loop runs about log n times and the inner loop runs up to n times.

```
i ← 1
WHILE i < n DO
   FOR j ← 0 TO i − 1 DO
      Audit(Request[j])            // basic operation
   END FOR
   i ← i × 2
END WHILE
```

Module M3 – Product Search: Two versions exist. Version S1 scans the unsorted catalogue sequentially until the product is found. Version S2 searches a sorted product index by comparing

the key with the middle entry and discarding half the index after each comparison; its comparison count satisfies T(n) = T(n/2) + 1, T(1) = 1.

Module M4 – Recommendation Engine: The engine splits the customer–product data into four parts of size n/2 (because of replicated partitions) and merges the partial results in linear time: T(n) = 4T(n/2) + n. A proposed redesign removes the replication, giving T(n) = 2T(n/2) + n.

Module M5 – Sales Analytics Engine: The analytics engine splits the sales data into halves recursively and sorts the records by revenue at every level, giving T(n) = 2T(n/2) + n log n. Separately, profiling shows that the nightly report generator performs $f(n) = 3n^2 + 10n \log_2 n + 50$ operations.

(a) Understand the Problem: For each of the five modules, identify the input size and the basic operation that dominates the running time, explain where it occurs, and state how you expect its execution count to grow as n increases. Justify each choice briefly.

(b) Analyse the Iterative Modules: For M1, derive the exact number of comparisons as a function of n and state the time and space complexity. For M2, derive the exact number of Audit operations, decide whether the team member's Θ(n log n) claim is correct and give the tight bound. Verify your M2 result for n = 1,024.

(c) Solve a Recurrence by Substitution: Solve the recurrence for search version S2 using backward substitution, showing every intermediate step, and derive the exact closed form. State the time complexity and the space complexity of both an iterative and a recursive implementation.

(d) Apply the Master Theorem: For M4, apply the Master Theorem to both the current design and the proposed redesign, clearly identifying a, b, f(n), n^(log_b a) and the case used, and explain what happens to the running time when the data doubles. For M5, determine whether any of the three standard cases applies to T(n) = 2T(n/2) + n log n; if not, derive the complexity using the extended Master Theorem or a recursion tree.

(e) Analyse Best, Average and Worst Cases: For search versions S1 and S2, identify the input condition that produces the best, average and worst case and give the number of comparisons in each. For S1, derive the average number of comparisons when the probability that the searched product exists is p, and evaluate it for n = 1,000,000 and p = 0.5.

(f) Prove an Asymptotic Bound: Using the formal definitions, prove that the report generator's cost $f(n) = 3n^2 + 10n \log_2 n + 50$ is $\Theta(n^2)$ by finding constants c_1, c_2 and n_0, and show that f(n) is not O(n).

(g) Scalability Decision: Estimate the number of basic operations and the running time of M1, M2, S2, M4 (both designs) and M5 for n = 1,000,000. Identify the modules that will fail to meet a nightly processing window of 10 minutes, and recommend, with justification, which modules must be redesigned and how.

Rubrics – Application-Based Question

Criteria	Weightage (%)	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Problem Context	20	Clearly understands the problem and accurately identifies all requirements	Good understanding with minor gaps	Basic understanding; some requirements unclear	Poor understanding or misinterpretation of the problem
Application of Concepts	30	Accurately applies appropriate concepts to solve complex problems	Applies relevant concepts correctly with minor errors	Applies basic concepts with limited accuracy	Fails to apply appropriate concepts
Problem-Solving Approach	20	Logical, well-structured, and efficient approach	Mostly logical approach with minor gaps	Basic approach with some inconsistencies	Illogical or unclear approach

Criteria	Weightage (%)	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Accuracy of Solution	20	Completely correct solution with proper justification	Mostly correct with minor errors	Partially correct solution	Incorrect or incomplete solution
Clarity	10	Clear, well-organized, and properly explained solution	Understandable with minor clarity issues	Limited clarity and explanation	Poorly presented and difficult to understand

Mark Evaluation:

Criteria	Weightage (%)	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Problem Context	20				
Application of Concepts	30				
Problem-Solving Approach	20				
Accuracy of Solution	20				
Clarity	10				
Total Marks					

Code of Conduct

I **KAVIYA.G(192524281)** certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student:  Date: 11-10-2026

ANSWERS

CSA06 Design and Analysis of Algorithms (CO1)

Approach and Conventions

The running time of an algorithm is measured here by counting its **basic operation**, the statement that is executed most often. Counting operations instead of seconds makes the analysis independent of the machine; the count is converted to real time only at the end, using the given server speed: **time = number of operations ÷ 10^8 per second**.

Throughout, log means $\log_2$. The catalogue size is $n = 10^5$ today and $n = 10^6$ after the expansion. The nightly window of 10 minutes is 600 s, which allows at most $600 \times 10^8 = \mathbf{6 \times 10^{10}}$ **operations**. Where a recurrence is solved exactly, T(1) = 1 is used.

(a) Understanding the Problem

For every module the input size, the dominant basic operation, the place where it occurs and the expected growth are identified below, each with its justification.

Module M1 – Duplicate-Listing Detector

Input size: n, the number of products in the catalogue.

Basic operation: the comparison in the IF statement, that is, comparing the title and the seller of Product[i] with those of Product[j].

Where it occurs: inside the inner FOR loop, so it is executed once for every pair (i, j) with i < j.

Expected growth: the number of pairs among n products is about $n^2/2$, so the count grows **quadratically**: doubling the catalogue makes the work four times larger.

Justification: the IF test runs for every pair, whereas MarkDuplicate(j) runs only when a real duplicate is found. The test is therefore the most frequently executed statement.

Module M2 – API Rate Limiter

Input size: n, the upper limit of the request window (the WHILE loop continues while i < n).

Basic operation: Audit(Request[j]), which the question itself marks as the basic operation.

Where it occurs: inside the inner FOR loop, which runs i times in each pass of the WHILE loop while i doubles after every pass (1, 2, 4, 8, …).

Expected growth: the pass sizes form a geometric series whose total is about the size of the last term, so growth is about n, that is, **linear**, even though there are log n passes.

Justification: Audit is the only work done inside the nested loops, and every other statement runs at most once per pass.

Module M3 – Product Search (versions S1 and S2)

Input size: n, the number of entries in the catalogue (S1) or in the sorted index (S2).

Basic operation: the comparison of the search key with a catalogue or index entry.

Where it occurs: in S1, once for each entry examined during the sequential scan; in S2, once per halving step, when the key is compared with the middle entry.

Expected growth: S1 may examine all n entries, so it grows **linearly**. S2 discards half of the index after each comparison, so it needs about $\log_2 n$ comparisons and grows **logarithmically**.

Justification: in both versions the comparison decides what happens next, and nothing else is repeated as often.

Module M4 – Recommendation Engine

Input size: n, the number of records in the customer–product data.

Basic operation: the merge step, which processes each of the n records once per call (the "+ n" in the recurrence), together with the recursive calls on n/2 records.

Where it occurs: at every node of the recursion, after the sub-problems return.

Expected growth: in the current design every call creates four sub-problems of half the size, so the total work doubles from one level to the next and grows as $\mathbf{n^2}$. In the redesign every call creates only two, so each level costs n and the log n levels give $\mathbf{n \log n}$.

Justification: the number of sub-problems (4 against 2) is what separates the two designs, and it is exactly what the replication removes.

Module M5 – Sales Analytics Engine (and the report generator)

Input size: n, the number of sales records.

Basic operation: the comparison made while sorting the records by revenue; a call on n records costs n log n.

Where it occurs: at every level of the recursion, because the records are sorted again after each split.

Expected growth: there are log n levels and each costs roughly n log n or less, so the total grows like $\mathbf{n \log^2 n}$, slightly faster than n log n.

Report generator: the input size is again n and the basic operation is the elementary operation counted by $f(n) = 3n^2 + 10n \log_2 n + 50$. For large n the term $3n^2$ dominates, so its growth is **quadratic**.

The table summarises part (a).

Module	Input size	Basic operation	Growth
M1	n products	Title/seller comparison in the IF test	$\approx n^2/2$ (quadratic)
M2	n (window limit)	Audit(Request[j])	$\approx n$ (linear)
M3 – S1 / S2	n entries	Key comparison	n (linear) / $\log_2 n$ (logarithmic)
M4	n records	Merge step plus recursive calls	n^2 (current) / n log n (redesign)
M5	n records	Comparison during sorting at every level	$\approx n \log^2 n$
Report generator	n records	Elementary operation counted by f(n)	$\approx 3n^2$ (quadratic)

(b) Analysing the Iterative Modules

Module M1 – exact number of comparisons

The basic operation is the IF comparison. To count it, the loops are analysed from the inside outwards.

Step 1 – inner loop. For a fixed value of i, the variable j takes the values $i + 1, i + 2, \ldots, n - 1$. The number of values is $(n - 1) - (i + 1) + 1 = \mathbf{n - i - 1}$, so the IF test is executed $n - i - 1$ times for that i.

Step 2 – outer loop. The outer loop takes $i = 0, 1, \ldots, n - 2$. For $i = 0$ there are $n - 1$ comparisons, for $i = 1$ there are $n - 2$, and so on until $i = n - 2$, which gives 1. Therefore

$$C(n) = (n - 1) + (n - 2) + \ldots + 2 + 1$$

Step 3 – evaluate the sum. This is the sum of the first $n - 1$ positive integers. Using $1 + 2 + \ldots + m = m(m + 1)/2$ with $m = n - 1$:

$$\mathbf{C(n) = (n - 1)n/2 = (n^2 - n)/2}$$

Time complexity. $C(n) = n^2/2 - n/2$. For large n the term $n^2/2$ dominates and constants are ignored, so the time complexity is $\mathbf{\Theta(n^2)}$. The best, average and worst cases are the same, because the loops always compare every pair no matter what the data contain.

Space complexity. Only the loop counters i and j are used besides the input, so the extra space is $\mathbf{\Theta(1)}$.

Meaning for the platform. For $n = 10^5$ the routine makes $4{,}999{,}950{,}000 \approx 5 \times 10^9$ comparisons, which takes about 50 s. For $n = 10^6$ it makes $499{,}999{,}500{,}000 \approx 5 \times 10^{11}$ comparisons, which takes about 5,000 s ≈ 83 minutes. The catalogue is only ten times larger, yet the night job is a hundred times longer. Memory is not the problem; time is.

Module M2 – exact number of Audit operations

Step 1 – number of outer passes. The value of i is 1, 2, 4, …, that is $i = 2^t$ in pass $t = 0, 1, 2, \ldots$. The loop continues while $2^t < n$, that is, while $t < \log_2 n$. If k denotes the number of passes, then $k = \text{ceil}(\log_2 n)$; when n is a power of two, $k = \log_2 n$.

Step 2 – work in one pass. In pass t the inner loop runs for $j = 0, 1, \ldots, i - 1$, which is exactly $i = 2^t$ values. So pass t performs $\mathbf{2^t}$ Audit operations. It does not perform n of them.

Step 3 – total work. Adding the passes gives a geometric series:

$$A(n) = 2^0 + 2^1 + 2^2 + \ldots + 2^{k-1} = (2^k - 1)/(2 - 1) = \mathbf{2^k - 1}$$

Step 4 – express in n. Since $k = \text{ceil}(\log_2 n)$, we have $n \le 2^k < 2n$, hence $n - 1 \le A(n) \le 2n - 2$. When n is a power of two, $\mathbf{A(n) = n - 1}$. Both bounds are proportional to n, so the tight bound is $\mathbf{\Theta(n)}$.

Verification for n = 1,024. Here $\log_2 1{,}024 = 10$, so there are 10 passes with $i = 1, 2, 4, \ldots, 512$ (when i becomes 1,024 the condition $i < n$ fails and the loop stops). The Audit operations are counted below.

Pass t	$i = 2^t$	Audit operations	Running total
0	1	1	1
1	2	2	3
2	4	4	7
3	8	8	15
4	16	16	31
5	32	32	63
6	64	64	127
7	128	128	255

Pass t	$i = 2^t$	Audit operations	Running total
8	256	256	511
9	512	512	**1,023**

The total is $1{,}023 = 2^{10} - 1 = n - 1$, which agrees with the formula.

Is the team member's claim correct? No. The claim says the outer loop runs about log n times and the inner loop "up to n times", and multiplies the two to get n log n. The first factor is right, but the inner loop runs n times only in the very last pass; in the earlier passes it runs 1, 2, 4, … times, far fewer. Multiplying the largest values of both loops overestimates the work. Adding the actual pass sizes, as done above, shows that the last pass alone is about half of the total. For n = 1,024 the claim predicts $n \log_2 n = 10{,}240$ operations, whereas only 1,023 occur, about ten times fewer. So n log n is a valid upper bound, O(n log n), but it is not tight and cannot be Θ. **The correct tight bound is Θ(n).**

Meaning for the platform. For $n = 10^6$, M2 performs 1,048,575 operations, about 0.01 s, so the rate limiter scales comfortably.

(c) Solving the Recurrence of Search Version S2 by Backward Substitution

Binary search compares the key with the middle entry (one comparison) and then continues on one half of the index, giving **$T(n) = T(n/2) + 1$ with $T(1) = 1$**. Backward substitution repeatedly replaces the T-term on the right by the recurrence itself until the pattern is clear. Assume $n = 2^m$ so that repeated halving ends exactly at 1.

Step 1. Start from the recurrence:

$$T(n) = T(n/2) + 1$$

Step 2. Replace T(n/2) by T(n/4) + 1:

$$T(n) = [T(n/4) + 1] + 1 = T(n/4) + 2$$

Step 3. Replace T(n/4) by T(n/8) + 1:

$$T(n) = [T(n/8) + 1] + 2 = T(n/8) + 3$$

Step 4. Replace T(n/8) by T(n/16) + 1:

$$T(n) = T(n/16) + 4$$

Step 5 – general pattern. After k substitutions, each of which halves the argument and adds 1:

$$T(n) = T(n/2^k) + k$$

Step 6 – stop at the base case. The recursion ends when $n/2^k = 1$, that is, $2^k = n$, so $k = \log_2 n$. Substituting and using T(1) = 1:

$$T(n) = T(1) + \log_2 n = \mathbf{1 + \log_2 n}$$

For an n that is not a power of two the exact closed form is **$T(n) = \text{floor}(\log_2 n) + 1$**.

Check. T(1) = 1, T(2) = T(1) + 1 = 2, T(4) = 3, T(8) = 4, and the formula gives $1 + \log_2 2 = 2$, $1 + \log_2 4 = 3$, $1 + \log_2 8 = 4$. For n = 1,024 the formula gives 11 comparisons.

Time complexity. $T(n) = 1 + \log_2 n$ is **Θ(log n)** for both implementations, because the same number of halving steps is made. The best case is a single comparison, O(1).

Space complexity. The two implementations differ in memory, as summarised below.

Implementation	Time	Space	Explanation
Iterative	$\Theta(\log n)$	$\Theta(1)$	A loop with the variables low, high and mid is reused in every step; no extra memory grows with n.
Recursive	$\Theta(\log n)$	$\Theta(\log n)$	Each recursive call is kept on the call stack until it returns. The depth is $\log_2 n + 1$, about 20 frames for $n = 10^6$.

Meaning for the platform. Even for 10^6 products a search needs about 20 comparisons, so product search does not limit scalability.

(d) Applying the Master Theorem

The Master Theorem solves recurrences of the form T(n) = aT(n/b) + f(n), where a ≥ 1 sub-problems of size n/b are solved and f(n) is the cost of dividing and combining. The deciding quantity is n^(log_b a), the total work of the leaves of the recursion tree, which is compared with f(n):

Case 1: if f(n) = O(n^(log_b a − ε)) for some ε > 0 (leaves dominate), then T(n) = Θ(n^(log_b a)).

Case 2: if f(n) = Θ(n^(log_b a)) (both are equal), then T(n) = Θ(n^(log_b a) log n).

Case 3: if f(n) = Ω(n^(log_b a + ε)) for some ε > 0 and a·f(n/b) ≤ c·f(n) for some c < 1 (the root dominates), then T(n) = Θ(f(n)).

Module M4 – current design: T(n) = 4T(n/2) + n

Step 1 – identify the parameters: a = 4, b = 2 and f(n) = n.

Step 2 – compute n^(log_b a): $\log_2 4 = 2$, so n^(log_b a) = n^2.

Step 3 – compare: f(n) = n = O(n^(2 − ε)) with ε = 1 > 0, so f(n) is polynomially smaller than n^2. This is **Case 1**.

Step 4 – conclude: T(n) = Θ(n^2). Solving exactly with T(1) = 1 gives $T(n) = 2n^2 - n$.

Module M4 – proposed redesign: T(n) = 2T(n/2) + n

Step 1: a = 2, b = 2, f(n) = n.

Step 2: $\log_2 2 = 1$, so n^(log_b a) = n.

Step 3: f(n) = n = Θ(n) = Θ(n^(log_b a)), so both are of the same order. This is **Case 2**.

Step 4: T(n) = Θ(n log n). The exact solution is $T(n) = n \log_2 n + n$.

What happens when the data doubles

For the current design, $T(2n)/T(n) = (2n)^2/n^2 = 4$, so **doubling the data multiplies the time by four**. For the redesign, $T(2n)/T(n) = 2n \log(2n) / (n \log n) = 2(1 + 1/\log_2 n)$, which is only slightly above 2 (about 2.1 for $n = 10^6$), so **doubling the data roughly doubles the time**. Over the planned growth from 10^5 to 10^6 products the current design becomes 100 times slower, whereas the redesign becomes only about 12 times slower. At $n = 10^6$ the redesign needs about 2.1×10^7 operations against about 2×10^{12}, which is nearly 100,000 times fewer. Removing the replication therefore changes the complexity class itself.

Module M5 – T(n) = 2T(n/2) + n log n

Here a = 2, b = 2, f(n) = n log n and n^(log_b a) = n. Each case is tested in turn.

Case 1 needs f(n) = O(n^(1 − ε)), that is, f smaller than n. It fails, because n log n is larger than n.

Case 2 needs f(n) = Θ(n). It fails, because f(n)/n = log n grows without bound.

Case 3 needs f(n) = Ω(n^(1 + ε)) for some ε > 0. It fails, because n log n / n^(1 + ε) = log n / n^{ε} tends to 0 for every ε > 0; a logarithm never overtakes a power of n.

Conclusion: none of the three standard cases applies, because f(n) exceeds n^(log_b a) only by a logarithmic factor, which falls in the gap between Cases 2 and 3.

Extended Master Theorem. If f(n) = Θ(n^(log_b a) · $\log^k$ n) with k ≥ 0, then T(n) = Θ(n^(log_b a) · $\log^{k+1}$ n). Here f(n) = n log n = n · $\log^1$ n, so k = 1, and

$$\mathbf{T(n) = \Theta(n \log^2 n)}$$

Verification by recursion tree. Let L = $\log_2$ n. At level i there are 2^i sub-problems of size n/2^i, each costing (n/2^i) log(n/2^i), so the whole level costs 2^i · (n/2^i) · (L − i) = n(L − i). The levels i = 0, 1, …, L − 1 therefore cost nL, n(L − 1), …, n·1, and the n leaves cost n in all.

Level i	Nodes	Size per node	Cost per node	Cost of the level
0	1	n	n log n	nL
1	2	n/2	(n/2) log(n/2)	n(L − 1)
2	4	n/4	(n/4) log(n/4)	n(L − 2)
⋮	⋮	⋮	⋮	⋮
L − 1	n/2	2	2	n
L (leaves)	n	1	1	n

Adding the levels:

$$T(n) = n[L + (L - 1) + \ldots + 1] + n = n \cdot L(L + 1)/2 + n = \tfrac{1}{2} n \log^2 n + \tfrac{1}{2} n \log n + n$$

The dominant term is ½ n $\log^2$ n, which confirms **T(n) = Θ(n $\log^2$ n)**. The extra factor of log n compared with ordinary merge sort comes from sorting again at every level instead of merging.

(e) Best, Average and Worst Cases of the Search Versions

Version S1 – sequential search

Best case: the product is the first entry of the catalogue. Exactly **1 comparison** is needed.

Worst case: the product is the last entry or is not in the catalogue at all. All **n comparisons** are needed.

Average case with probability p that the product exists. Two outcomes are possible.

Step 1 – product present (probability p). Each of the n positions is equally likely, so position i has probability p/n, and finding it costs i comparisons.

Step 2 – product absent (probability 1 − p). The whole catalogue is scanned, costing n comparisons.

Step 3 – expected value. Adding the two contributions:

$$A(n) = \Sigma\, i \cdot (p/n) \text{ (for } i = 1 \text{ to } n) + n(1 - p) = (p/n) \cdot n(n + 1)/2 + n(1 - p)$$

$$\mathbf{A(n) = p(n + 1)/2 + n(1 - p)}$$

Sanity check. If $p = 1$ the formula gives $(n + 1)/2$, the usual successful-search average; if $p = 0$ it gives n, the cost of a failed scan.

Step 4 – evaluation for n = 1,000,000 and p = 0.5:

$A = 0.5 \times (1{,}000{,}001)/2 + 1{,}000{,}000 \times 0.5 = 250{,}000.25 + 500{,}000 =$ **750,000.25 comparisons**

This is about $0.75n$, so one search takes about 7.5 ms and the cost is $\Theta(n)$.

Version S2 – binary search on the sorted index

Best case: the key is exactly the middle entry, found at the first comparison: **1 comparison**.

Worst case: the key is absent, or lies at the deepest level of the search, so the index is halved until one entry remains: **floor($\log_2 n$) + 1 comparisons**, which is 20 for $n = 10^6$ (this equals the closed form of part (c)).

Average case: the comparisons form a binary decision tree in which about half of all entries lie at the deepest level and a quarter one level above, so a successful search needs about **$\log_2 n - 1$** comparisons, roughly 19 for $n = 10^6$; an unsuccessful search needs about $\log_2 n$. The complexity is $\Theta(\log n)$ in both cases.

The results are summarised below.

Version	Case	Input condition	Comparisons ($n = 10^6$)
S1	Best	Key is the first entry	1
S1	Average	Key present with probability $p = 0.5$	750,000.25
S1	Worst	Key is last or absent	1,000,000
S2	Best	Key is the middle entry	1
S2	Average	Key found at a typical depth	≈ 19
S2	Worst	Key at deepest level or absent	20

(f) Proving the Asymptotic Bound for the Report Generator

Definition. $f(n) = \Theta(g(n))$ means that there are positive constants c_1, c_2 and n_0 such that $c_1 \cdot g(n) \le f(n) \le c_2 \cdot g(n)$ for every $n \ge n_0$. Here $f(n) = 3n^2 + 10n \log_2 n + 50$ and $g(n) = n^2$. We must find a lower and an upper bound of the form constant $\times n^2$.

Step 1 – lower bound. For $n \ge 1$ both $10n \log_2 n$ and 50 are non-negative, so dropping them can only decrease the value:

$$f(n) \ge 3n^2 \text{ for all } n \ge 1, \text{ so } c_1 = 3$$

Step 2 – upper bound. Each term is bounded by a multiple of n^2 for $n \ge 8$. First, $\log_2 n \le n$, so $10n \log_2 n \le 10n^2$. Second, $n^2 \ge 64 > 50$ when $n \ge 8$, so $50 \le n^2$. Therefore

$$f(n) = 3n^2 + 10n \log_2 n + 50 \le 3n^2 + 10n^2 + n^2 = 14n^2, \text{ so } c_2 = 14 \text{ and } n_0 = 8$$

Step 3 – conclusion. For all $n \ge 8$, $3n^2 \le f(n) \le 14n^2$. With $c_1 = 3$, $c_2 = 14$ and $n_0 = 8$ the definition is satisfied, so **$f(n) = \Theta(n^2)$**.

Numerical check. For n = 8: f = 3(64) + 10(8)(3) + 50 = 482, and 192 ≤ 482 ≤ 896. For n = 1,000: f ≈ 3,099,708, and 3,000,000 ≤ 3,099,708 ≤ 14,000,000. Both hold.

Step 4 – f(n) is not O(n). Suppose, for contradiction, that f(n) = O(n). Then there are constants c > 0 and n_0 with f(n) ≤ c · n for all n ≥ n_0. But f(n) ≥ 3n², so 3n² ≤ c · n, which simplifies to n ≤ c/3. This must hold for every n ≥ n_0, yet it is false for any n larger than both n_0 and c/3. This contradiction shows that no such constant exists. Equivalently, f(n)/n ≥ 3n, which tends to infinity, so f grows strictly faster than n. Hence **f(n) ∉ O(n)**.

Meaning for the platform. The report generator is truly quadratic: when the catalogue grows tenfold, the report takes about a hundred times longer.

(g) Scalability Decision for n = 1,000,000

The number of operations is obtained from the formulas derived above, with T(1) = 1, and the time from time = operations ÷ 10^8. The limit is 6 × 10^{10} operations (600 s).

M1. C(n) = n(n − 1)/2 = 499,999,500,000 ≈ 5.0 × 10^{11} operations. Time ≈ 4,999.995 s ≈ 5,000 s ≈ **83.3 minutes**, about 8 times the window.

M2. A(n) = 2^{20} − 1 = 1,048,575 ≈ 1.05 × 10^6 operations (20 passes, because 2^{20} is the first power of two not below 10^6). Time ≈ **0.0105 s**.

S2. floor($\log_2 10^6$) + 1 = 19 + 1 = **20 comparisons** per search. Time ≈ **0.2 microseconds**.

M4, current design. T(n) = 2n² − n ≈ 2 × 10^{12} operations. Time ≈ 20,000 s ≈ **5.6 hours**, about 33 times the window.

M4, redesign. T(n) = n $\log_2$ n + n = 19,931,569 + 1,000,000 ≈ 2.1 × 10^7 operations. Time ≈ **0.21 s**.

M5. T(n) ≈ ½ n $\log^2$ n + ½ n log n + n ≈ 1.99 × 10^8 + 1.0 × 10^7 + 1.0 × 10^6 ≈ 2.1 × 10^8 operations. Time ≈ **2.1 s**.

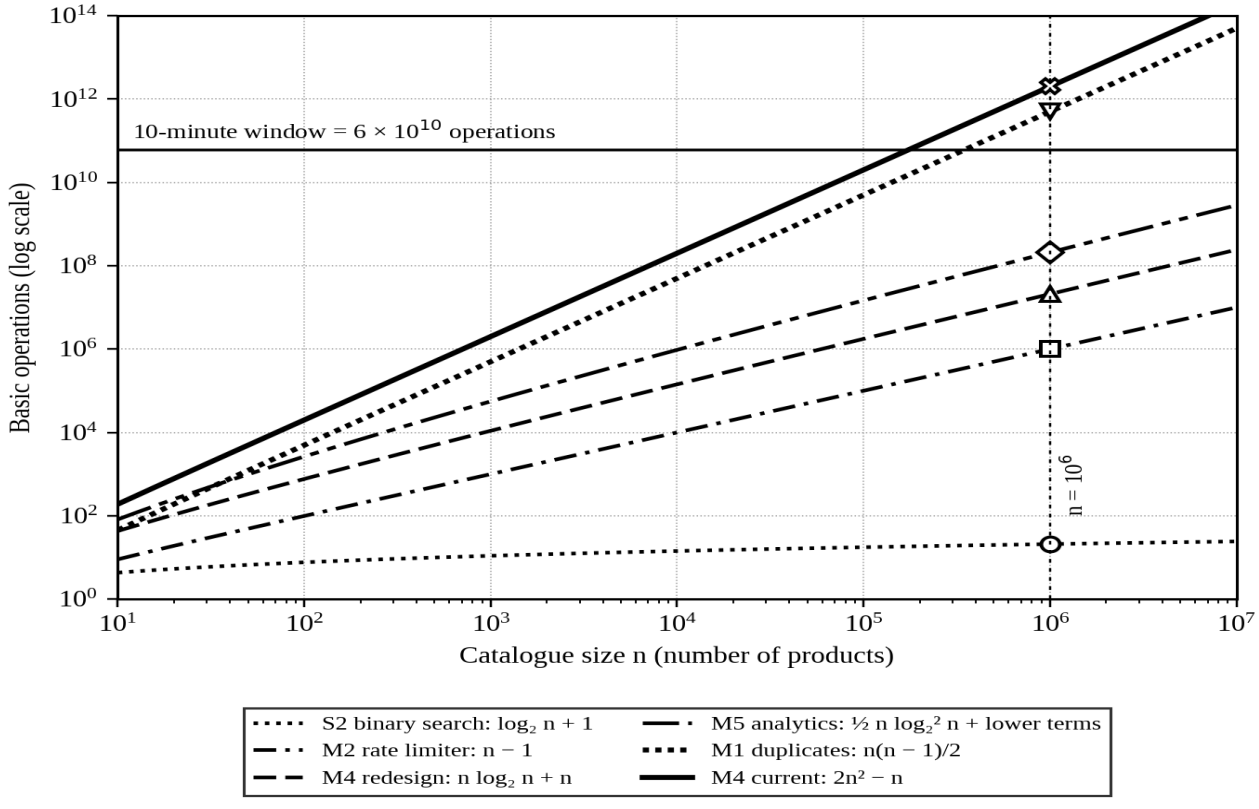


Figure 1. Operation counts against catalogue size n (log–log scale). Markers show $n = 10^6$; the horizontal line is the 10-minute window.

Module	Complexity	Operations at $n = 10^6$	Running time	Within 10 min?
M1	$\Theta(n^2)$	$\approx 5.0 \times 10^{11}$	≈ 83.3 min	**No**
M2	$\Theta(n)$	$\approx 1.05 \times 10^6$	≈ 0.0105 s	Yes
S2 (one search)	$\Theta(\log n)$	20	≈ 0.2 µs	Yes
M4 – current	$\Theta(n^2)$	$\approx 2 \times 10^{12}$	≈ 5.6 h	**No**
M4 – redesign	$\Theta(n \log n)$	$\approx 2.1 \times 10^7$	≈ 0.21 s	Yes
M5	$\Theta(n \log^2 n)$	$\approx 2.1 \times 10^8$	≈ 2.1 s	Yes